

[ESPOCRM How To Guide/](#)

Claude is responding

I need to understand security in ESPOCRM

Two quick things before I dig in:

1. **CLAUDE.md**: Should I read the one in `crmbuilder`, the one in `ClevelandBusinessMentoring`, both, or neither for this session?
2. **Scope**: Security in EspoCRM is layered. Where do you want to start?
 - The access control model end-to-end (Users → Teams → Roles → record/field permissions), top-down
 - Just **Roles** (the scope/level matrix, action permissions per entity)
 - Just **Teams** (record-level visibility, default teams, team-based assignment)
 - **Field-level** read/edit security
 - **Assignment permissions** (who can assign records to whom)
 - **Authentication** (passwords, 2FA, LDAP/OAuth/SAML, API keys)
 - **Portal users** and external access
 - Something else specific (e.g., a CBM scenario you're trying to model)

My instinct, given the CBM How-To Guide context, is to start with the access control model top-down so the pieces fit together, then drill into whichever layer matters most for CBM. But I'll wait for your direction.

I don't know where to start.

4:27 PM

Fair enough — let me give you the mental model top-down. Once this clicks, the rest of EspoCRM security is just detail.

The question being answered

EspoCRM security answers one question, over and over: *can this user do this action on this record?* Four inputs determine the answer — who the user is, what teams they belong to, what roles they have, and what's on the record itself.

One escape hatch worth flagging up front: **System Administrators bypass all of this**. When you test permissions, never test as admin — you'll see everything regardless of how broken your config is. Create a test user and impersonate them, or test with a real non-admin account.

The three building blocks

Users are individuals. Each user has a primary team, optional additional teams, a default assigned user (usually themselves), and one or more roles. Users can also have a *default team set* — any record they create is automatically stamped with those teams.

Teams are groupings. They do two jobs: (1) record-level access keys off them, and (2) roles can be attached to teams, so every team member inherits those roles. A user can belong to many teams, and a record can be assigned to many teams.

Roles are the permission definitions. A role is essentially a matrix: for each entity (Contact, Account, Meeting, your custom entities), what actions are allowed and at what level.

The action × level matrix — the core

For every entity, a role defines five actions: **Create, Read, Edit, Delete, Stream**. Create is binary yes/no. The other four each take one of four levels:

- **All** — any record of this entity, regardless of team or ownership.
- **Team** — only records where the user shares at least one team with the record.
- **Own** — only records where the user is the `assignedUser`.
- **No** — never.

So `Read=Team`, `Edit=Own` on Contacts means "I see every Contact in my team's pipeline, but I can only edit ones assigned to me."

When a user has multiple roles, **EspoCRM takes the most permissive level across them**. Roles add up; they don't restrict each other. This catches people out — you can't use a second role to *subtract* permissions, only to grant more.

Field-level security

Beneath the entity-level matrix, you can lock down individual fields per role: read and edit, independently. Everyone might read a Contact, but only managers see the `salaryExpectation` field. Field-level rules can only *further restrict* what the entity-level matrix already permits — they can't grant access the entity-level denies.

Assignment permissions

Separate axis from CRUD. Assignment permissions answer: "who can I set as `assignedUser` on a record I'm creating or editing?" Options are All / Team / No. This matters whenever you have a delegation model — e.g. a coordinator can assign mentors to clients, but mentors can only assign records to themselves.

Portal users — parallel universe

External users (a mentee logging into a self-service portal, say) live in a completely separate scheme: **Portal Users + Portal Roles**, scoped to a specific **Portal**. Structurally the role mechanics are the same, but they're configured in different places and don't share definitions with internal roles. Don't conflate the two.

That's the whole skeleton. The CBM-relevant places to drill in next, in the order they usually matter:

1. **Team structure for CBM** — should teams be per mentor-mentee pairing, per program cohort, per staff function, or some mix? This decision drives most of the rest.
2. **Role inventory** — at minimum you'll need Staff, Mentor, Portal-Mentee, Admin. What does each see and edit?
3. **Default Team Set mechanics** — exactly how records get auto-stamped with teams at creation, and what that means for workflow design.
4. **Concrete walk-through** — pick one entity (e.g. Contact or a CBM-specific entity) and we trace exactly how a Mentor's view differs from a Staff's.

My recommendation: start with #4, because abstract permission talk gets slippery fast. Pick the entity that's central to CBM's daily work and we'll walk through who sees what, then back into the team and role structures that produce that result. Which entity? (*Contact? Mentee? Session?*)

Lets go with option 4

4:31 PM

Pick the entity and I'll build the walk-through around it. Three good candidates given what's central to CBM's daily work:

Contact — foundational, but it's a mixed bag at CBM (mentors, mentees, prospects, donors all live here). Security gets interesting because different roles need to see different *subsets* of Contacts.

Meeting (1:1 sessions and group sessions) — high daily traffic, natural team-based visibility (a mentor sees their own sessions; staff see all sessions in a program), and it ties directly to the Zoom integration work.

Notes (your custom entity in design) — richest for security because of the sensitivity gradient: some notes are private staff observations about a mentee, some are shared between mentor and staff, some are visible to the mentee. Also actively in design, so the walk-through would directly inform decisions you're about to make.

My recommendation: **Notes**. The security model has to handle the broadest range of visibility patterns, so once you can express Notes correctly in EspoCRM's role/team model, everything else is easier. And it answers questions you'll need answered soon anyway.

Confirm Notes, or pick Contact / Meeting / something else.

notes

4:36 PM

Architected concrete Notes security walkthrough with realistic CBM scenarios

Architected concrete Notes security walkthrough with realistic CBM scenarios

Good pick. Here's the walk-through. I'll use a working assumption for team structure so the example stays concrete — flagged below so you can revisit it later.

Working assumption: team structure

For this walk-through, CBM has two kinds of teams:

- **Staff** — one team, contains every CBM staff member.
- **Pairing teams** — one team per active mentor-mentee pairing, e.g. "Pairing: Jane Smith ↔ Bob Jones." Members are the mentor, the assigned staff coordinator, and any others involved.

This is a clean model but it's a *decision*, not a given. We'll revisit "is pairing-per-team the right grain" separately.

The cast

- **Jane Smith** — volunteer mentor. In team **Pairing: Jane ↔ Bob** only.
- **Carlos Reyes** — staff coordinator on this pairing. In team **Staff** and team **Pairing: Jane ↔ Bob**.
- **Priya Patel** — staff coordinator on a *different* pairing. In team **Staff** and team **Pairing: Priya ↔ someone else**.
- **Bob Jones** — the mentee, portal user. Set aside for now; covered at the end.

The three Notes we want to control

Each Note has a **teams** field (which teams can see it) and an **assignedUser** (the author/owner).

Note	Author	Content	Should be visible to
A	Jane (mentor)	"Discussed Bob's business plan revisions"	Jane, Carlos, other staff if needed
B	Carlos (staff)	"Bob disclosed financial hardship — consider pausing intensives"	Carlos, other staff. Not Jane.
C	Carlos (staff)	"Bob hit Q1 milestone — share at next session"	Jane, Carlos, all staff, eventually Bob

The interesting one is **B**. Carlos shares a team with Jane (**Pairing: Jane ↔ Bob**). How do we keep Note B from her?

The mechanism

Two levers do all the work:

1. **The role's Notes matrix** — what level of access each role gets generally.
2. **The teams field on each Note record** — which teams' members can apply that access to *this specific* note.

Here's a workable role config:

Staff Role, Notes entity:

- Create: ✓
- Read: **All** (or *Team* if you want staff to only see notes for pairings they touch — for CBM probably All)

- Edit: **Own**
- Delete: **Own**
- Stream: All

Mentor Role, Notes entity:

- Create: ✓
- Read: **Team**
- Edit: **Own**
- Delete: **Own**
- Stream: Team

Now watch what happens, given how the records get *stamped* with teams:

- **Note A** gets teams = [Pairing: Jane ↔ Bob], assignedUser = Jane. Jane sees it (Team match). Carlos sees it (Team match, he's in the pairing team). Priya doesn't see it — she's not in that pairing team and her role is Read=All only because she's *Staff*... wait, that breaks. Let me re-check.

Actually with Staff Role = Read=All, Priya *would* see Note A. If that's not what you want, Staff = Read=Team is the answer, and staff need to be added to the relevant pairing teams to see those notes. That's a real CBM decision: **do all staff see all notes by default, or only notes for pairings they're attached to?** Park that — it's a downstream consequence.

Assuming Staff = Read=All for simplicity:

- **Note A** (teams = [Pairing: Jane ↔ Bob]) — Jane sees it (Team match on the pairing). All staff see it (Read=All).
- **Note B** (teams = [Staff] only — *not* the pairing team) — Jane does *not* see it (no team match, and her role is Read=Team only). All staff see it.
- **Note C** (teams = [Pairing: Jane ↔ Bob, Staff]) — Jane sees it. All staff see it. Bob doesn't see it yet (different system — see portal section).

The "aha"

The teams field on a record is the access control knob. Staff using EspoCRM aren't really thinking about "permissions" when they write Note B — they're thinking about *which teams should this note be visible to?* and choosing accordingly. The role config sets the ceiling; the per-record team assignment decides who actually gets in.

This means two operational requirements:

1. **Staff need training** on what teams to attach to sensitive notes. Default Team Set on a user only helps for the *common* case; sensitive notes need conscious choice.
2. **You probably want a workflow or validation** that catches the easy mistake — e.g. a Note flagged as "Confidential" should not have any pairing teams attached, only *Staff*. Worth a downstream decision.

Where field-level security adds something

The above handles *whole-record* visibility. Field-level is for when everyone can see the record but some fields are restricted. Example: a Note has a `confidentialFlag` field, and you want only staff to be able to set that flag (not mentors). Field-level edit permission on `confidentialFlag` for the Mentor Role = No solves it. Field-level doesn't help with Note B — that's whole-record hiding, which is the teams mechanism.

Bob the mentee — portal side

Bob isn't an internal user. He logs into a portal. His access is governed by a **Portal Role**, configured separately. A typical setup:

- Bob is linked to a Contact record (his own).
- Portal Role gives Read=Own (or Team, depending) on Notes.
- Notes you want Bob to see get him added either via the `assignedUser`, a portal-team mechanism, or a direct relationship to his Contact.

This is enough complexity that I'll save the portal walk-through for its own pass — but the headline is: **mentee visibility is a parallel system, configured separately, and you should not try to make internal team-based logic reach him.**

Summary in one paragraph

Roles define the ceiling — what *kinds* of access a user could have to Notes. Teams on each Note record decide who actually gets in. Field-level locks down specific fields within an otherwise-visible record. Portal users live in their own parallel role system.

Next step options:

1. **Capture this in the CBM How-To guide** under a new "Security model" section, with the three-Notes example as the worked illustration.
2. **Decide CBM's team structure** for real (pairing-per-team vs. cohort vs. hybrid) now that you've seen how teams drive everything.
3. **Decide the Staff Read level** (All vs. Team) — the open question I parked above.
4. **Move on to a different entity** (Meeting, Contact) using the same pattern.

My recommendation: **#1 first** while it's fresh, then **#2**. Confirm and I'll draft the How-To section.

never mind... Let me give you a scenario and you best explain how to use it.

4:40 PM

Go ahead — describe the scenario. Who the users are, what they should and shouldn't see/do, and which entities are involved. I'll work out the role + team configuration to produce that outcome.

We have a typical entity: Company. However, we have some types of companies that we need to collect sensitive information, so we create a second entity called company_sensitive. We link the two records with a One-To-One relationship. We want all users to see all company base records. and we want to assign the right to see the sensitive information to a few individuals. Each company with sensitive information may have different users who can see it.

4:44 PM

Good scenario — and the architecture decision you've already made (splitting into a separate entity) is the right one. Let me verify EspoCRM's mechanism specifics before I commit to a recommendation.

Validated architecture and synthesized Collaborators feature solution

Validated architecture and synthesized Collaborators feature solution

Confirmed — EspoCRM has exactly the right primitive for this scenario, and I want to give you the concrete answer.

The architecture decision is correct

Splitting into a separate `CompanySensitive` entity is the *necessary* move. EspoCRM's field-level security is per-role, not per-record — you cannot say "User X sees the `bankBalance` field on Acme but not on Beta." Splitting converts an unsolvable field-level-per-record problem into a solvable record-level problem.

The right mechanism: Collaborators

EspoCRM 9 introduced a feature called **Collaborators** that fits your "different users for different records" requirement exactly. Here's how it works:

Enabling it — In Entity Manager for `CompanySensitive`, enable Collaborators. This automatically creates a link-multiple field called Collaborators and a "Shared" bool filter. The admin then needs to add the Collaborators field to the side panel layout (Administration > Entity Manager > `CompanySensitive` > Layouts > Side Panel Fields). This last step catches people out — if you skip it, the feature works but nobody can see the field to manage it. [GitHub](#)

What it grants — Users added as collaborators get read and stream access to the record, provided their role's access level for that entity is something other than "no". Important nuance: Collaborators grants **read access, not edit access**. So a collaborator can see a `CompanySensitive` record but not modify it unless they get edit rights another way. [GitHub](#)

Auto-behaviors — The creator of a record is auto-added as a collaborator, and the assigned user is automatically added too, so reassigning a record doesn't accidentally cut off the previous owner. [GitHub](#)

Constraints — Adding a user as a collaborator requires the adder to have Assignment Permission and edit access to the record. Portal users cannot be added as collaborators. [GitHub](#)

Concrete role configuration

General User Role (everyone):

- Company: Read = **All**, Edit/Create/Delete per your normal rules
- CompanySensitive: Read = **Own**, Edit = No, Create = No, Delete = No

That Read = Own is the key. Combined with the Collaborators feature, a user gets read access to a CompanySensitive record only if they are the assigned user *or* listed as a collaborator. For every other sensitive record, they see nothing — not even the existence in list views.

Sensitive Info Coordinator Role (the few people who can manage all sensitive records — set up access, edit data):

- CompanySensitive: Read = All, Edit = All, Create = Yes, Delete = All, Stream = All
- Plus Assignment Permission = at least Team (required to add collaborators)

Coordinators see every CompanySensitive record, can populate the data, and decide who else gets to see it by editing the Collaborators field.

If authorized users also need to *edit* sensitive info

Collaborators alone grants read-only. If a few designated users per company need to actively maintain sensitive data, you have two options:

1. **Make them the assigned user** of that specific CompanySensitive record. Their role's Edit = Own then lets them edit it. Limitation: one assigned user per record.
2. **Enable the Multiple Assigned Users feature** on CompanySensitive (separate from Collaborators — it's a different Entity Manager toggle). This replaces the single assignedUser with an assignedUsers link-multiple. All assigned users get Own-level access, including edit if the role allows. Better fit if multiple people need edit rights per record.

If you just need different *viewers* per record — and a small coordinator group does the editing — stick with plain Collaborators.

Operational considerations

Who manages the collaborator list? The Coordinator role above, in practice. They open a CompanySensitive record and add the right users to the Collaborators field. This is a manual, deliberate act per record, which is appropriate for sensitive-data access decisions.

Relationship visibility from Company side. When User X views a Company record they don't have sensitive access to, the related-records panel for CompanySensitive will show its header but no linked record will appear (or the panel will be empty). The *existence* of a sensitive record is somewhat leaked through the panel header. If you want to fully hide that, either omit the panel from the side layout for non-Coordinators (via different layouts per role) or live with the small leak. Worth a small test to confirm exactly what non-authorized users see.

Audit trail. Stream on `CompanySensitive` will record who was added/removed as a collaborator over time, giving you an audit trail of access grants. Worth keeping Stream enabled for this reason.

One thing to confirm

Do users granted access to a `CompanySensitive` record need to *edit* it, or only *view* it?

- **View only** → plain Collaborators. Coordinators handle all editing.
- **Edit too** → enable Multiple Assigned Users on the entity instead of (or in addition to) Collaborators.

Tell me which, and I'll lock in the final role + entity configuration and we can capture this pattern in the How-To guide as "Per-Record Sensitive Data Pattern."

Any user who is allowed to access the sensitive record can edit the information. We would prefer not to leak the fact that there is sensitive information. Please expand on how we can make the panel not appear if the user does not have access to the data?

5:02 PM

Good — both questions have distinct answers. Let me handle them in order, then bring them together.

Let me check what EspoCRM supports for conditional panel visibility before I commit.

Synthesized role-based field security and access control strategies

Synthesized role-based field security and access control strategies

Both questions have clean answers. Putting it all together:

Edit access for authorized users → Multiple Assigned Users

Since access-equals-edit, **Collaborators is the wrong tool** (it grants read-only). The right tool is **Multiple Assigned Users**, which you enable per-entity in Entity Manager. This replaces the single `assignedUser` field with a multi-user `assignedUsers` field. Every user in that list gets Own-level access to the record — read *and* edit, governed by the role's Own-level settings.

So the role for "any user who might be granted access to sensitive records":

- `CompanySensitive`: Read = **Own**, Edit = **Own**, Delete = Own (or No), Create = No

Granting access to a specific company's sensitive record = adding the user to that record's `assignedUsers`.

Hiding the panel → field-level security on the link

This is the central trick. EspoCRM lets you set field-level read access on **link fields**, not just regular fields. So the relationship between Company and `CompanySensitive` (the `companySensitive` link field on Company) can itself be hidden per role.

When a role has field-level read = No on that link:

- The relationship panel on the Company detail view does not render at all (no header, no empty state)
- The link field is omitted from the API response
- Filters and search by that field are unavailable
- It's a clean hide, not just visual suppression

EspoCRM's role configuration supports hiding both full entities and single fields at the bottom of the role config, and field-level security lets you specify read and edit access independently per field via the Field Level section of a Role record. [EspocrmEspocrm](#)

The role architecture that produces no leak

The mental model: **two roles, additive**. Every user has the base role; only authorized people get the second one stacked on top. Roles merge to the most permissive level, so adding the second role *grants* access without breaking the first.

Role 1 — "Standard User" (everyone has this):

Scope	Setting
Company entity	Read=All, Edit/Create/Delete per normal rules
Company → field companySensitive (the link)	Read = No
CompanySensitive entity	Read=No, Edit=No, Create=No, Delete=No
Effect: a Standard-only user sees the Company entity normally, has no idea CompanySensitive exists. The panel doesn't render. Search, list views, related-records — nothing about sensitive data leaks anywhere.	

Role 2 — "Sensitive Authorized" (added to users who need access to at least one sensitive record):

Scope	Setting
Company → field companySensitive (the link)	Read = Yes
CompanySensitive entity	Read=Own, Edit=Own, Create=No, Delete=No
Effect: the panel renders on Company detail views. But because CompanySensitive Read=Own combined with Multiple Assigned Users means "only records where I'm in assignedUsers," the panel only shows the records they're authorized for. On Companies where a sensitive record exists but they're not authorized, the panel renders empty — indistinguishable from "no sensitive record exists at all." Among the authorized pool, that's an acceptable level of disclosure: they know sensitive records exist <i>in the system</i> , but cannot tell whether any specific Company has one they can't see.	

Role 3 — "Sensitive Coordinator" (the small group who grants/revokes access):

Scope	Setting
Company → field companySensitive	Read = Yes
CompanySensitive entity	Read=All, Edit=All, Create=Yes, Delete=All
Assignment Permission	At least Team (required to modify assignedUsers)

Effect: sees and manages all sensitive records, including the `assignedUsers` field that determines who else has access.

Operational flow

1. Coordinator creates a `CompanySensitive` record, links it to the relevant `Company`, populates the data.
2. Coordinator adds the authorized users to the `assignedUsers` field.
3. Coordinator (or admin) ensures each of those users has the Sensitive Authorized role assigned. *This is the small ongoing operational task — granting the role when someone first becomes eligible to see any sensitive data.*
4. Authorized user logs in, sees the panel on `Companies`, sees their authorized records, can edit them.
5. Standard users see nothing about sensitive data anywhere.

One trap to avoid

When you test this, **never test as Admin** — admins bypass all access control and will see everything regardless. Create a `test_standard` user (only Standard role) and a `test_authorized` user (both roles, added to `assignedUsers` of one specific sensitive record) and verify each sees only what they should.

Summary configuration

- Entity Manager → `CompanySensitive` → enable **Multiple Assigned Users**
 - Entity Manager → `Company` → relationship to `CompanySensitive` (one-to-one, link field `companySensitive`)
 - Role: **Standard User** — Field Level on `Company`: `companySensitive` Read=No, Edit=No. Scope: `CompanySensitive` = No on everything.
 - Role: **Sensitive Authorized** — Field Level on `Company`: `companySensitive` Read=Yes. Scope: `CompanySensitive` Read=Own, Edit=Own.
 - Role: **Sensitive Coordinator** — Field Level on `Company`: `companySensitive` Read=Yes. Scope: `CompanySensitive` Read=All, Edit=All, Create=Yes, Delete=All.
 - Users: every user gets Standard. Authorized people additionally get Sensitive Authorized. The few coordinators get Sensitive Coordinator (and Standard for normal `Company` access).
-

Next step: I recommend capturing this as a named pattern in the CBM EspoCRM How-To guide — call it the "**Sensitive Sub-Entity Pattern**" — with the three-role config, the field-level-on-link trick, and the test procedure. It's reusable for any future case where some records need restricted, per-record access lists. Confirm and I'll draft the section.

nope. Your explanations are not clear enough. What does "Role: Standard User — Field Level on Company: companySensitive Read=No, Edit=No. Scope: CompanySensitive = No on everything." even mean?

10:58 PM

Fair complaint — I compressed three separate EspoCRM concepts into one line. Let me unpack properly. The problem is you can't apply my line without first understanding how the Role configuration screen is *organized*.

What the Role edit screen actually looks like

When you open a Role in EspoCRM (Administration > Roles > click a role), you see a form with three distinct sections, in this order:

Section 1 — Scope Level Permissions (the big table)

This is the entity-by-entity matrix. One row per entity in the system (Contact, Account, Meeting, your custom CompanySensitive, etc.). For each row, columns are:

- **Access:** enabled / disabled / not-set
- **Create:** yes / no
- **Read:** All / Team / Own / No
- **Edit:** All / Team / Own / No
- **Delete:** All / Team / Own / No
- **Stream:** All / Team / Own / No

This controls whether and how the role can interact with each entity *as a whole*.

Section 2 — Field Level Permissions (smaller section below)

Here you drill *into specific fields within an entity* and override read/edit access just for those fields. The workflow: click a plus icon next to an entity name, pick a field from the dropdown, then set:

- **Read:** Yes / No
- **Edit:** Yes / No

Field level can only restrict further — it can never grant access the scope level denies.

Section 3 — Permissions (system-wide settings)

Things like Assignment Permission, Export Permission, etc. Not entity-specific. Not relevant to this scenario right now.

The connecting insight you need

In EspoCRM, a relationship between two entities appears as a "field" on each side.

When you create the one-to-one relationship between Company and CompanySensitive, EspoCRM automatically creates two link fields:

- On the **Company** entity: a field named `companySensitive` (points to the related `CompanySensitive` record)
- On the **CompanySensitive** entity: a field named `company` (points back)

These link fields can be controlled by Field Level Permissions exactly like any regular field. **That is the mechanism that hides the relationship panel.**

Now re-reading the line that confused you

"Role: Standard User — Field Level on Company: `companySensitive` Read=No, Edit=No.
Scope: `CompanySensitive` = No on everything."

That line is shorthand for **two configuration actions** inside the Standard User role:

Action 1 — in the Scope Level Permissions table

Find the `CompanySensitive` row. Set its Access to **disabled** (or equivalently: Create=No, Read=No, Edit=No, Delete=No, Stream=No — all "No").

What this does: The Standard User role grants zero access to the `CompanySensitive` entity. The user cannot see, list, create, or interact with it anywhere. `CompanySensitive` effectively doesn't exist for this user — no nav menu item, no list view, no search results, nothing.

Action 2 — in the Field Level Permissions section

Click the plus icon next to "Company" (to add a field-level override on the Company entity). From the field dropdown, pick `companySensitive`. Set its Read=No and Edit=No.

What this does: For this role, the `companySensitive` link field on Company records is hidden. The relationship panel doesn't render on the Company detail view. The field doesn't appear in API responses, search, filters, or anywhere else. Looking at a Company record, the Standard user sees no trace that a sensitive record could even be related.

Why both actions are necessary

Action	Hides	Without it
1 (Scope = No)	The <code>CompanySensitive</code> entity itself, everywhere	User might find <code>CompanySensitive</code> via nav menus, list views, search
2 (Field Level on the link)	The relationship from Company's side	The panel still renders on Company detail views, leaking that sensitive records exist
You need both. Action 1 alone leaves the panel visible on Company. Action 2 alone leaves the entity browsable elsewhere.		

Does this clear it up? If yes, I'll re-unpack the Sensitive Authorized and Sensitive Coordinator roles the same way — section by section, action by action — so each role is something you can actually click through in the UI, not shorthand.

Lets re-write these instructions with real entity names. That will make it clearer. Let use the Company and ClientProfile entities where the ClientProfile is the sensitive data.

11:31 PM

Good — concrete names will help. Here it is, rewritten clean, with `Company` (everyone sees all records) and `ClientProfile` (sensitive data, one-to-one with `Company`).

One-time setup (do these first, before configuring roles)

Step 1 — Create the ClientProfile entity

Administration > Entity Manager > Create Entity. Name it `ClientProfile`. Set the type to Base or whatever fits. Save.

Step 2 — Create the one-to-one relationship to Company

Administration > Entity Manager > `ClientProfile` > Relationships > Create Link.

- **Relationship type:** One-to-One Right (or whichever direction makes `Company` the "parent")
- **Foreign Entity:** `Company`
- **Link (this side):** `company` — this creates a field on `ClientProfile` pointing to the related `Company`
- **Link Foreign (other side):** `clientProfile` — this creates a field on `Company` pointing to the related `ClientProfile`

EspoCRM auto-creates both link fields and auto-adds the related panel to the `Company` detail layout.

Step 3 — Enable Multiple Assigned Users on ClientProfile

Administration > Entity Manager > `ClientProfile` > Edit. Find the "Multiple Assigned Users" checkbox (sometimes labeled "Assigned Users" with a "multiple" option). Enable it. Save.

This replaces the single `assignedUser` field with an `assignedUsers` link-multiple field. Every user listed in `assignedUsers` on a `ClientProfile` record will get Own-level access to it.

Role 1: Standard User

Who gets this role: Everyone. It's the baseline.

Purpose: Grants normal access to `Company`. Completely hides `ClientProfile` and any sign of its existence.

Administration > Roles > Create. Name it `Standard User`.

Section 1 — Scope Level Permissions

Find the rows for `Company` and `ClientProfile`. Set them as follows:

Entity	Access	Create	Read	Edit	Delete	Stream
Company	enabled	yes	All	(your normal rule)	(your normal rule)	All
ClientProfile	disabled	—	—	—	—	—

Setting ClientProfile to **disabled** means this role has zero access to it: no list view, no nav menu, no search, no API. It's invisible.

Section 2 — Field Level Permissions

Click the plus icon next to Company. From the field dropdown, select `clientProfile`. Set:

Field	Read	Edit
Company → <code>clientProfile</code>	No	No

This hides the `clientProfile` link field on Company records for this role. The related-record panel doesn't render on the Company detail view. The field doesn't appear in API responses, filters, or search.

Result

A user with only this role: sees every Company record as normal, sees no panel for ClientProfile on Company, has no idea ClientProfile exists anywhere in the system.

Role 2: Sensitive Authorized

Who gets this role: Any user who needs access to at least one ClientProfile record. Stacked *on top of* Standard User — you assign both roles to the user, and EspoCRM merges them to the more permissive setting.

Purpose: Lets the user see the ClientProfile entity exists, see ClientProfile records they're assigned to (via `assignedUsers`), and edit those records.

Administration > Roles > Create. Name it **Sensitive Authorized**.

Section 1 — Scope Level Permissions

Entity	Access	Create	Read	Edit	Delete	Stream
ClientProfile	enabled	no	Own	Own	no	Own

Read = **Own** combined with Multiple Assigned Users means: "I can see ClientProfile records where I'm listed in `assignedUsers`." Edit = **Own** lets those same users edit those records.

Leave Company alone here (don't override it). The Standard User role already grants Company access.

Section 2 — Field Level Permissions

Add an override on Company for the `clientProfile` field, setting:

Field	Read	Edit
Company → <code>clientProfile</code>	Yes	No

Why this is necessary: the Standard User role hides this field with Read=No. Without an explicit Yes here, role merging would still hide it (No is more restrictive, but the merge rule is *more permissive wins* — so Yes here overrides the No from Standard).

Result

A user with Standard + Sensitive Authorized:

- Sees Company normally.
 - Sees the ClientProfile-related panel on Company detail views.
 - On Companies where they are in the `assignedUsers` of the related ClientProfile: the record appears, they can open and edit it.
 - On Companies where a ClientProfile exists but they aren't authorized: the panel renders empty — indistinguishable from "no ClientProfile exists for this Company."
 - Cannot create or delete ClientProfile records; only the Coordinator does that.
-

Role 3: Sensitive Coordinator

Who gets this role: The small group of people who create ClientProfile records and decide who else gets access. Stacked on top of Standard User.

Purpose: Full management of all ClientProfile records, including modifying the `assignedUsers` list to grant/revoke access.

Administration > Roles > Create. Name it **Sensitive Coordinator**.

Section 1 — Scope Level Permissions

Entity	Access	Create	Read	Edit	Delete	Stream
ClientProfile	enabled	yes	All	All	All	All

This role sees and can manage every ClientProfile record in the system.

Section 2 — Field Level Permissions

Same as Sensitive Authorized — add an override on Company for `clientProfile`:

Field	Read	Edit
Company → clientProfile	Yes	Yes

Section 3 — Permissions

Find **Assignment Permission** and set it to **All** (or at minimum Team).

This is required for the Coordinator to add users to the `assignedUsers` field of a ClientProfile. Without it, even though they can edit the record, they can't modify who else is assigned to it.

Result

A Coordinator (Standard + Sensitive Coordinator):

- Sees every ClientProfile record.
 - Can create new ClientProfile records and link them to a Company.
 - Can edit the assignedUsers field to grant or revoke access for specific users.
 - Can edit the sensitive data itself.
-

Assigning roles to users

Administration > Users > [pick a user] > Edit > Roles field.

- Most users: just **Standard User**.
 - People granted access to one or more ClientProfile records: **Standard User + Sensitive Authorized**. They still need to be added to the assignedUsers of the specific ClientProfile records they're authorized for — being in the role alone doesn't grant access to any records, only the *capability* to be granted access.
 - The handful of coordinators: **Standard User + Sensitive Coordinator**.
-

How sensitive access actually happens (the workflow)

1. A Coordinator opens or creates a ClientProfile record.
 2. Coordinator sets the company link on the ClientProfile to the relevant Company.
 3. Coordinator populates the sensitive data fields.
 4. Coordinator opens the assignedUsers field and adds the specific users who should have access. (These users must already have the **Sensitive Authorized** role on their user record. If they don't, an admin adds it first.)
 5. Those users can now see the ClientProfile panel populated on the Company detail view, open the record, and edit the data. All other users see nothing.
-

How to test

Create three test users:

- **test_standard** — only **Standard User** role.
- **test_authorized** — **Standard User + Sensitive Authorized**. Add them to assignedUsers of exactly one ClientProfile record (linked to, say, "Acme Inc").
- **test_coordinator** — **Standard User + Sensitive Coordinator**.

Log in as each (use an incognito window — never test as admin, who bypasses everything) and verify:

User	Acme Inc detail view	Beta Corp detail view (also has ClientProfile, but test_authorized is not on it)	ClientProfile in nav/lists
test_standard	No ClientProfile panel	No ClientProfile panel	Not visible

User	Acme Inc detail view	Beta Corp detail view (also has ClientProfile, but test_authorized is not on it)	ClientProfile in nav/lists
test_authorized	Panel visible, ClientProfile record visible and editable	Panel visible, empty	Not visible in nav (Read=Own restricts list views to own records)
test_coordinator	Panel visible, record visible and editable	Panel visible, record visible and editable	Visible, full list

If any test case fails, the role config has an error — most commonly a Field Level override missing or set wrong.

Read through this and tell me where it's still unclear or wrong. Once you're satisfied, the natural next step is capturing it in the CBM How-To Guide as the **Sensitive Sub-Entity Pattern** section.

Please explain (Your Normal Rule)?

11:36 PM

Good catch — that was a lazy placeholder. Let me explain what I meant and then give you the actual options.

What "(your normal rule)" was doing

The original scenario you described constrained two things explicitly:

1. **All users see all Company records** → that's why Read is locked to **All** in my table.
2. **Sensitive data lives in ClientProfile, hidden from most users** → that's what the rest of the pattern handles.

But you never said anything about who's allowed to **edit, delete, or create** Company records. Those decisions are orthogonal to the sensitive-data pattern — they're regular CRM access decisions about how editable your Company directory should be. So I left placeholders rather than make up an answer.

The actual options for each column

These are the standard EspoCRM levels for the Company row in the Scope table:

Create (yes / no):

- **yes** — anyone with this role can add a new Company to the system.
- **no** — only roles that explicitly allow it (e.g., admin, a "Sales" role) can create Companies.

Edit (All / Team / Own / No):

- **All** — anyone can edit any Company.
- **Team** — only Companies whose **teams** field includes a team the user belongs to.
- **Own** — only Companies where the user is in **assignedUser(s)**.
- **No** — read-only; cannot edit any Company.

Delete (All / Team / Own / No):

- Same options as Edit. Usually set more restrictively than Edit.

Common patterns and which fits CBM

Three typical patterns for a Company-like entity:

Open editing (low-control orgs): Create=yes, Edit=All, Delete=No. Anyone can add or edit any Company; nobody but admin can delete. Simple, good for small teams where data quality is enforced socially.

Owned editing (sales-style): Create=yes, Edit=Own, Delete=Own. You can add a Company, and only the person assigned to it can edit it. Good when each Company "belongs to" a single account manager.

Team editing (cohort/program-style): Create=yes, Edit=Team, Delete=No. You can edit any Company in your team's portfolio but not others. Good when staff work in distinct portfolios.

Read-only directory (high-control): Create=No, Edit=No, Delete=No. Only specific roles (e.g., a "Data Steward" role) can modify Companies. Everyone else just reads.

For CBM specifically, my guess would be **owned editing** or **team editing** — staff/coordinators can edit Company records for the mentee businesses they work with, but not casually edit Companies they don't touch. But that's a real decision you should make, not something I should infer.

Plugging it back into the table

Once you decide, the Standard User row in Section 1 becomes concrete. For example, if you pick "team editing":

Entity	Access	Create	Read	Edit	Delete	Stream
Company	enabled	yes	All	Team	No	All
ClientProfile	disabled	—	—	—	—	—

Or if "owned editing":

Entity	Access	Create	Read	Edit	Delete	Stream
Company	enabled	yes	All	Own	Own	All
ClientProfile	disabled	—	—	—	—	—

The ClientProfile row stays the same regardless — it's disabled either way.

Decision: Which Company edit pattern do you want — Open, Owned, Team, or Read-only? Once you tell me, I'll rewrite the role tables with the actual values filled in, so the whole spec is concrete.

I assume that we could set up a team role in addition to a Standard role? So the standard role would allow a user to edit companies that they were an owner (IE they created), or they could edit companies if they were on the "Data Integrity Team". Otherwise, they would not be able to edit a company even if they saw an error that needed correction.

11:42 PM

Yes — that composition works exactly as you've described, and it's the right way to think about it. Let me explain the mechanism and the concrete config.

The merging rule that makes this work

When a user has multiple roles in EspoCRM, the roles are merged and **the most permissive level wins**. So if Standard User says Edit=Own and Data Integrity says Edit=All, a user who has both ends up with Edit=All on Company. A user with only Standard stays at Edit=Own. This is exactly the additive layering you want.

Two ways to give someone the Data Integrity role

This is the EspoCRM feature you may not have realized exists: **a role can be attached to a team, and every member of that team automatically inherits the role**.

- **Direct assignment:** Administration > Users > [user] > Roles field → add Data Integrity role to specific users.
- **Team-based assignment (better for your case):** Create a Data Integrity Team, attach the Data Integrity role to that team's Roles field, then just add or remove users from the team membership. The role follows team membership automatically.

The team-based approach is cleaner because membership management lives in one place — you don't have to touch each user record when someone joins or leaves the data integrity group.

The updated configuration

Updated Role 1: Standard User (modified)

Change the Company row in Section 1 (Scope Level Permissions):

Entity	Access	Create	Read	Edit	Delete	Stream
Company	enabled	yes	All	Own	No	All
ClientProfile	disabled	—	—	—	—	—

Section 2 (Field Level on `Company.clientProfile`) unchanged: Read=No, Edit=No.

Effect: Every user can see every Company but can only edit Companies where they are the `assignedUser` (typically because they created the record — see note below).

New Role: Data Integrity

Administration > Roles > Create. Name it `Data Integrity`.

Section 1 — Scope Level Permissions:

Entity Access Create Read Edit Delete Stream

Company enabled not-set not-set **All** not-set not-set

Leave everything else as **not - set** — the role only needs to *add* edit power. The user's other roles (Standard User) handle the rest, and "not-set" means "don't override what other roles say."

Section 2 — Field Level Permissions: none. This role isn't about sensitive data; ignore ClientProfile entirely.

Section 3 — Permissions: none specific.

New Team: Data Integrity Team

1. Administration > Teams > Create. Name it **Data Integrity Team**.
2. In the team's edit form, find the **Roles** field. Add **Data Integrity** to it.
3. Save.
4. Add users to the team via the team's members panel, or via each User > Teams field.

What users experience

User profile	Roles	Effect on Company
Regular staff	Standard User	Reads any Company. Edits only Companies they're the assignedUser of.
Data integrity volunteer	Standard User + Data Integrity (via team)	Reads any Company. Edits any Company.
Regular staff who sees an error on a Company they don't own	Standard User	Cannot edit. Has to flag it to someone on the Data Integrity Team.

One important nuance: "owner" = assignedUser, not createdBy

You said "companies that they were an owner (IE they created)." EspoCRM's **Edit=Own** checks the **assignedUser** field, **not the createdBy field**. These are usually the same for fresh records — by default, when you create a Company, you become its assignedUser automatically. But if a Company is later reassigned to someone else, Edit=Own access transfers with the assignment. The original creator loses edit access.

Two implications:

1. If you want "the creator can always edit, even after reassignment," that's a different model and requires either keeping the creator in a Collaborators or **assignedUsers** field (which means enabling Multiple Assigned Users on Company), or a more custom approach.
2. If you want assignment-based editing (the default Edit=Own behavior), then "owner" means "currently assigned," which may or may not match your intent.

For most CRMs the assignment-based model is what people actually want — it tracks who currently has responsibility, not who originally entered the data. If that fits, no change needed.

Composition recap

Three additive roles, applied to users in this pattern:

- **Standard User** — every user has this. Read all Companies. Edit own. ClientProfile completely hidden.
- **Sensitive Authorized** — added to users who can see at least one ClientProfile. Unlocks the ClientProfile panel and lets them see/edit ClientProfile records they're assigned to.
- **Data Integrity** — attached to the Data Integrity Team. Added to a user's effective permissions by adding them to that team. Unlocks Edit=All on Company.
- **Sensitive Coordinator** — added to the small group who manages ClientProfile records and assignedUsers.

A single user can have any combination. Effects merge to the most permissive setting.

One question before we proceed: for the "owner" semantic — do you want **assignment-based** (default Edit=Own behavior, transfers with reassignment) or **creator-based** (whoever first entered the record always retains edit)? The first is essentially free; the second needs a different mechanism. Tell me which, and I'll lock the final spec.

We will use assignment based owner semantic.

11:50 PM

Locked. EspoCRM's default Edit=Own checks assignedUser, which transfers cleanly with any reassignment — exactly the semantic you want. No extra configuration needed beyond what's already in the spec.

The spec is complete

Quick recap of everything we built for the **Sensitive Sub-Entity Pattern**:

Entity setup

- ClientProfile entity created
- One-to-one relationship to Company (creates clientProfile link field on Company, company link field on ClientProfile)
- Multiple Assigned Users enabled on ClientProfile

Four roles

- **Standard User** — Company: Read=All, Edit=Own, Create=yes, Delete=No / ClientProfile: disabled / Field Level on Company.clientProfile = Read=No, Edit=No
- **Sensitive Authorized** — ClientProfile: Read=Own, Edit=Own / Field Level on Company.clientProfile = Read=Yes (rest not-set)
- **Sensitive Coordinator** — ClientProfile: Read=All, Edit=All, Create=yes, Delete=All, Stream=All / Field Level on Company.clientProfile = Read=Yes / Assignment Permission = All
- **Data Integrity** — Company: Edit=All (everything else not-set)

One team

- Data Integrity Team, with the Data Integrity role attached to its Roles field

User assignment pattern

- Everyone: Standard User
- Layer Sensitive Authorized on anyone who can ever be granted access to any ClientProfile
- Add to Data Integrity Team to grant elevated Company edit rights
- Sensitive Coordinator goes to the small group who manages ClientProfile access

Test plan

- Three test users (test_standard, test_authorized, test_coordinator), tested in incognito windows — never tested as admin

Next step

Capture this in the CBM EspoCRM How-To Guide as a new **Security > Sensitive Sub-Entity Pattern** section.

Please upload the current CBM-EspoCRM-HowTo .docx. I'll draft the addition as a surgical insert — preserving existing content, applying the output standards (revision control table, change log entry, "Last Updated" timestamp in MM-DD-YY HH:MM format), and showing you the proposed text before committing. After your review, I'll regenerate the .docx and you can commit/push at session end as usual.

Whoa... Not just yet.

11:51 PM

Understood — stepping back. What would you like to do next?

Lets expand our scope a little. We have standard users that can view all companies, and never see sensitive information. And we have users on the data integrity team that can view edit all companies and sensitive information. But we also have mentors that need to be able to access the sensitive information only for the companies that they are owners. They cannot see or edit sensitive information for non-owned companies.

11:55 PM

This is a clean expansion, but it introduces a new mechanism that's worth thinking through before we touch any configuration. Let me lay out the conceptual model first, then surface the one decision it forces.

The key insight: this is not a new role

Mentors need exactly the permissions that the **Sensitive Authorized** role already provides — Read=Own, Edit=Own on `ClientProfile`, with the `Company.clientProfile` link field unhidden. What's different is *how their access list gets populated*.

For the Coordinator-managed case we already designed:

- Coordinator manually picks users and adds them to `ClientProfile.assignedUsers`.

For the mentor case you just described:

- The current owner of the Company (the Company's `assignedUser`) should automatically be in the related `ClientProfile.assignedUsers`.

Both mechanisms write to the same field. The role doesn't care how a user got into the list — it just sees who's there. So mentors get the **Sensitive Authorized** role assigned to their user record, and the automation does the rest.

This means:

- **No new role definition** is needed for mentors.
- **A workflow rule** is needed to keep `ClientProfile.assignedUsers` in sync with the related `Company.assignedUser`.
- **Manual Coordinator grants still work** alongside the automation — the two mechanisms coexist on the same field.

The decision this forces

Here's the question that has to be answered before the workflow can be specified:

Plain-language question

When a Company is reassigned from one mentor to another, what should automatically happen to the previous mentor's access to that Company's sensitive data?

Concrete example

Carlos has been mentoring Acme Inc. for six months. The workflow has him in `Acme's ClientProfile.assignedUsers`, so he can see and edit Acme's sensitive data. In Q2, the staff reassigns Acme to Priya — `Acme.assignedUser` changes from Carlos to Priya. The workflow adds Priya to `ClientProfile.assignedUsers`. The question is: what happens to Carlos's entry?

The two options

Option A — Additive only. The workflow only ever *adds* users; never removes. Carlos remains in `assignedUsers` until a Coordinator manually removes him. Behavior the user notices: Carlos can still open Acme's `ClientProfile` and edit it after the reassignment, until cleanup happens.

Option B — Auto-revoke previous owner. The workflow detects that Carlos was Acme's previous `assignedUser` and removes him from `ClientProfile.assignedUsers` in the same step that adds Priya. Behavior the user notices: Carlos loses access to Acme the moment the reassignment is saved.

Why this matters

Sensitive data shouldn't linger with people whose role has changed — that's the privacy principle. But auto-revocation can be over-aggressive: if a Coordinator had explicitly added Carlos for some other reason (e.g., "Carlos is a domain expert we want to keep on this case even after reassignment"), Option B would silently remove him. The Coordinator wouldn't necessarily know to re-add.

There's also an audit-trail consideration: Option B makes access turnover automatic and silent; Option A makes every revocation an explicit, attributable action by a person.

Cost of the recommended option

Option A's cost is operational: stale access can linger if Coordinators don't clean it up. Mitigation is a Coordinator dashboard showing "ClientProfiles where `assignedUsers` includes someone who isn't the current Company owner" — making stale access visible and easy to clean up.

Recommendation

Option A. Explicit revocation is safer than implicit. Privacy is preserved through a visible cleanup workflow rather than silent automation that can over-remove.

Follow-on detail (not part of the core decision)

If you go with Option A, the additional pieces are:

- A list view or dashlet for Coordinators titled "Sensitive Records with Stale Access" filtering `ClientProfile` records where any `assignedUsers` entry doesn't match the related `Company.assignedUser`.
- A documented cleanup expectation — e.g., "Coordinator reviews stale access weekly."

If you go with Option B, the additional pieces are:

- A way to tag a user as "permanently authorized regardless of ownership" (probably a separate `permanentlyAuthorized` link-multiple on `ClientProfile`) so the workflow knows not to remove them.

Decision request

Approve **Option A (additive only, manual cleanup)**, or argue for **Option B (auto-revoke previous owner)**?

Once you decide, the workflow specification follows directly — it's a short EspoCRM workflow rule. We can also revisit whether mentors should be assigned the Sensitive Authorized role directly on their

user record, or via a "Mentors Team" with the role attached (same team-as-role-vehicle pattern we used for Data Integrity).

Hold on... Sensitive Authorized provides View All, and Edit All. Now View Own, Edit Own right?

11:59 PM

No — Sensitive Authorized has been Read=**Own**, Edit=**Own** throughout. You may be thinking of **Sensitive Coordinator**, which is the All/All role.

Quick reference on just the ClientProfile access levels across all four roles:

Role	ClientProfile Read	ClientProfile Edit
Standard User	(entity disabled — no access at all)	—
Sensitive Authorized	Own	Own
Sensitive Coordinator	All	All
Data Integrity	(not-set — doesn't grant ClientProfile access on its own)	—

So mentors getting the Sensitive Authorized role means Read=Own and Edit=Own on ClientProfile — they see and edit only ClientProfile records where they're in `assignedUsers`. The workflow we were just discussing populates `assignedUsers` based on Company ownership, which is exactly what produces the "mentor sees only their owned Companies' sensitive data" behavior.

A side note worth flagging: Data Integrity, as we defined it, only grants Company Edit=All — it does **not** grant ClientProfile access. So Data Integrity Team members can edit any Company, but if they need to also access sensitive data on all Companies, they'd additionally need the Sensitive Coordinator role. Worth deciding whether Data Integrity should also imply sensitive access, but that's a separate question from the mentor one.

Back to where we were: are we still going with **Option A (additive only, manual cleanup)** for the reassignment behavior, or do you want Option B?

I am leaning toward option A, but a few questions. If a mentor is already the owner of a company and clientprofile, they could add a new mentor and remove themselves?

12:07 AM

Good question — and the answer is **no, by default they can't**, because of a setting we haven't talked about yet but which is exactly designed to prevent this: **Assignment Permission**.

What Assignment Permission does

Assignment Permission is a system-level permission in Section 3 of the Role config (the "Permissions" section I mentioned but didn't dwell on). It has three levels:

- **No** — the user cannot modify any field that controls record ownership or access (the `assignedUser`, `assignedUsers`, `teams`, or `collaborators` fields on any record they touch).

- **Team** — they can only assign to users who share at least one team with them.
- **All** — they can freely modify ownership/access fields, assigning to any user.

This is enforced separately from regular Edit access. A user can have Edit=Own on a record — meaning they can change the data fields — without being able to touch the `assignedUsers` field. The field will render read-only for them.

What this means for your mentor scenario

In our spec so far, we only set Assignment Permission explicitly on **Sensitive Coordinator** (= All). For the other roles, we didn't address it — which means the system default (typically "no") applies.

For a mentor (Standard User + Sensitive Authorized):

- They can open the ClientProfile they own and **edit the sensitive data fields** (notes, financials, whatever's on the record).
- They **cannot modify assignedUsers** — the field appears but is read-only for them. They cannot add a new mentor, and they cannot remove themselves.
- They **cannot change Company.assignedUser** either — the same restriction applies to the Company side. They can't quietly reassign Acme to someone else.

So a mentor is "locked into" the access list as defined by the workflow and the Coordinator. Only Sensitive Coordinator can modify the access list on a ClientProfile, and only someone with appropriate Assignment Permission can reassign a Company.

Make this explicit in the spec

To leave nothing to defaults, set Assignment Permission explicitly per role:

Role	Assignment Permission
Standard User	No
Sensitive Authorized	No
Sensitive Coordinator	All
Data Integrity	(see note below)

Note on Data Integrity: This role grants Edit=All on Company. If we leave Assignment Permission off (= No), then Data Integrity Team members can fix data errors but can NOT reassign Company ownership. If we set it to All, they can also handle ownership changes. My recommendation is **No** — keep their job scoped to data quality, with ownership changes requiring a separate authority (admin or a designated reassignment role). But that's a separate decision you should make explicitly.

Related consideration: should mentors even see the `assignedUsers` field?

Even read-only, a mentor logged into a ClientProfile will see "this list contains Carlos and Priya and the Coordinator." That tells them who else has access to "their" client's sensitive data. Two views on this:

- **Show it** — transparency. The mentor knows who else can see this info, which can be reassuring or relevant.
- **Hide it** — minimize information leakage. Use Field Level Security on `ClientProfile.assignedUsers` with `Read=No` for the Sensitive Authorized role. Coordinator's view is unaffected.

Defaulting to "show it" is probably right (transparency is generally good), but worth a conscious choice.

So to summarize the answer to your question: **No, a mentor cannot add a new mentor and remove themselves**, as long as Assignment Permission is set to NO (or left at default) on the Sensitive Authorized role. This protects the integrity of the access list against unilateral mentor action — only Coordinators can manage who has sensitive access.

Want to lock in Assignment Permission = No on Standard User, Sensitive Authorized, and Data Integrity now, and keep the `assignedUsers` field visible? Or any of those calls different?

Not yet. So lets assume that we create a team called "Mentors", and we set the Assignment Permission to "Team". This would allow a mentor to add additional mentors, or transfer mentorship by removing themselves? We would trust our mentors to be able to assign other Mentors to a ClientRecord.

12:12 AM

Yes, this approach works exactly as you're describing, and it's actually a cleaner model for an environment where mentors are trusted peers. Let me walk through what it enables and where the edges are.

What becomes possible with Assignment Permission = Team + a Mentors team

For a mentor with Sensitive Authorized + Assignment Permission = Team, and membership in the Mentors team:

- **Add a fellow mentor as a co-mentor** on a `ClientProfile` they own — `assignedUsers` field becomes editable, and they can add any user who's also in the Mentors team.
- **Remove themselves** from `assignedUsers` to step away from a client.
- **Transfer mentorship** by adding the new mentor and then removing themselves — effectively a self-managed handoff.
- **Reassign the Company itself** (set `Company.assignedUser` to a different mentor) — same Team-level Assignment Permission applies to `Company` too, so they can reassign within the Mentors pool.

What's still restricted

The Team scope on Assignment Permission means mentors can only assign to *users in their own team*. So:

- A mentor **cannot add a non-mentor user** (a staff person, a Coordinator, a Data Integrity volunteer) to `assignedUsers`. The target user must be in the Mentors team. This is actually a safety feature — it keeps mentors from accidentally or maliciously granting access to people outside the trusted pool.
- A mentor **cannot remove a user who isn't in their teams**, or at least EspoCRM's behavior here is more conservative. If a Coordinator has specifically added an outside expert to a `ClientProfile`'s `assignedUsers`, the mentor likely can't yank that person out. (Worth empirically verifying when you build the test environment — EspoCRM's exact behavior on cross-team removal isn't documented as clearly as the add behavior.)

How the setup looks

1. **Create the Mentors team.** Administration > Teams > Create. Name: Mentors.
2. **Attach the Sensitive Authorized role to the team.** In the team's Roles field, add `Sensitive Authorized`. (Now adding a user to the team automatically gives them the role — single membership action, all permissions follow.)
3. **Update the Sensitive Authorized role** to set Assignment Permission = Team.
4. **Add all mentor users to the Mentors team.**

A user becoming a mentor = "add to Mentors team" — one step, role and assignment scope both come with it.

Interaction with the workflow (Option A)

This is the wrinkle worth thinking about. Under Option A (workflow only adds, never removes), here's what happens when Mentor A transfers Acme to Mentor B by self-managed handoff:

Step 1: A opens Acme's `ClientProfile`, adds B to `assignedUsers`. Now both A and B are listed. *Workflow doesn't fire — the trigger is Company changes, not ClientProfile edits.*

Step 2: A opens Acme's Company record and changes `Company.assignedUser` from A to B. *Workflow fires, adds B to `ClientProfile.assignedUsers` — but B is already there, so no change.*

Step 3: A still has Edit=Own on `ClientProfile` (still in `assignedUsers`). A removes themselves from `ClientProfile.assignedUsers`. Now only B remains.

Result: Clean transfer. Three actions by A. No Coordinator involvement needed.

If A skips Step 3, A stays in `assignedUsers` as a "co-mentor still on the case" — which might be desirable or not. The Coordinator's "stale access" dashboard would surface this (A in `assignedUsers` but not the Company owner), and the Coordinator could prompt cleanup.

One subtle caveat: the workflow can never fully self-correct

Under Option A, here's a scenario the workflow doesn't handle:

- A is the Company owner. A removes themselves from `ClientProfile.assignedUsers` (Step 3 above) but **doesn't** reassign the Company. So `Company.assignedUser` still = A, but A is no longer in `ClientProfile.assignedUsers`.
- A no longer has access to the `ClientProfile`.
- The workflow doesn't fire (no `Company.assignedUser` change) and won't re-add A.

This is "an owner can lock themselves out of the sensitive data without giving up ownership." Probably fine — it's an explicit user action with no harm done. But worth noting because it's a state the system permits.

My read

This configuration (Mentors team + Assignment Permission = Team) is a real upgrade for the mentor scenario. It treats mentors as trusted operators within a defined pool, which seems to match CBM's actual operating model. The Coordinator role retains the broader authority (cross-team assignments, including non-mentors); mentors handle peer-to-peer handoffs themselves without Coordinator bottleneck.

Recommendation: Adopt this. Use `Mentors` team as the vehicle for both the Sensitive Authorized role *and* the assignment scope, and set `Assignment Permission = Team` on Sensitive Authorized.

Want to lock this in along with the Option A workflow approach, or any more wrinkles to explore?

Nice work. But now I need to add the FINAL wrinkle....

12:18 AM

Bring it on. What's the wrinkle?

We have three types of sensitive information linked to three teams of users. We have Mentors, Sponsor Coordinators, and Partner Managers. There are three different entities that manage the sensitive information for each role, and the desired functionality is exactly the same as mentors, just for different types of information. And a user could be both a mentor and a partner manager.

12:20 AM

Validated pattern scalability and identified critical data model question

Validated pattern scalability and identified critical data model question

This is actually a clean generalization — the Sensitive Sub-Entity Pattern multiplies cleanly across as many sensitive types as you want, and EspoCRM's role-merging handles users in multiple teams without any special accommodation. A user in both Mentors and Partner Managers gets the Mentor

role's access to ClientProfile *and* the Partner Manager role's access to PartnerProfile, independently. The two grants don't interact because each role targets a different entity.

Before I sketch the full multi-instance configuration, I need to know how your data model is actually shaped. The setup differs noticeably between two possibilities.

One question: Are the three sensitive sub-entities all linked to the **same parent entity** (Company), or to **three different parent entities**?

- **Same parent (one Company entity).** A single Company might have a ClientProfile *and/or* a SponsorProfile *and/or* a PartnerProfile — three optional one-to-one relationships hanging off the same Company. Any Company can have any combination of the three.
- **Three different parents.** Client, Sponsor, and Partner are separate entities. Each has its own dedicated sensitive sub-entity (Client → ClientProfile, Sponsor → SponsorProfile, Partner → PartnerProfile). An organization is *one of* a Client, Sponsor, or Partner — not several.

The pattern works either way, but the role configurations and field-level-hiding details look different. Which matches your actual model?

Same Parent - Company

12:22 AM

Good — same Company parent keeps this very clean. The Sensitive Sub-Entity Pattern applies three times in parallel, with no interaction between the three instances. Here's the structure.

The pattern, parametrically

For each sensitive type, you set up:

- **A sensitive sub-entity** (one-to-one with Company, Multiple Assigned Users enabled)
- **A team** containing the users allowed to access that sensitive type
- **A role** attached to the team granting: Read=Own + Edit=Own on the sensitive entity, Field Level Read=Yes on the corresponding link field on Company, and Assignment Permission=Team
- **A workflow rule** that syncs Company.assignedUser → SensitiveEntity.assignedUsers when ownership changes

The three instances

Sensitive Entity	Link Field on Company	Team	Role Attached to Team
ClientProfile	clientProfile	Mentors	Client Sensitive Authorized
SponsorProfile	sponsorProfile	Sponsor Coordinators	Sponsor Sensitive Authorized
PartnerProfile	partnerProfile	Partner Managers	Partner Sensitive Authorized

Each instance is fully independent. The three workflows are independent. The three roles are independent.

The updated Standard User role

Standard User now hides **three** link fields on Company and disables **three** sensitive entities.

Scope Level Permissions changes:

Entity	Access
ClientProfile	disabled
SponsorProfile	disabled
PartnerProfile	disabled

Field Level Permissions on Company:

Field	Read	Edit
Company → clientProfile	No	No
Company → sponsorProfile	No	No
Company → partnerProfile	No	No

What users experience

The behavior emerges naturally from role merging — no special coordination needed:

User	Teams	What's visible on a Company detail view
Standard staff	(none of the three)	Company data. No sensitive panels at all.
Mentor only	Mentors	Company + ClientProfile panel (if owner). No Sponsor/Partner panels.
Partner Manager only	Partner Managers	Company + PartnerProfile panel (if owner). No Client/Sponsor panels.
Mentor <i>and</i> Partner Manager	Both	Company + ClientProfile panel + PartnerProfile panel (each conditional on ownership). No Sponsor panel.

The mechanism: each role independently unhides *exactly one* link field. A user in only Mentors has Read=Yes on `clientProfile` (from the Mentor role) but Read=No on the other two (from Standard User, never overridden). Role merging produces precisely the right per-user view without explicit coordination.

The decision this surfaces: Coordinator structure

We previously had a single "Sensitive Coordinator" role. With three sensitive types, this needs a call.

Plain-language question

Should sensitive-data administration be one unified role across all three types, or split into three per-type coordinator roles?

The two options

Option 1 — Three per-type coordinator roles

- "Client Sensitive Coordinator" — Read=All, Edit=All on ClientProfile only

- "Sponsor Sensitive Coordinator" — same for SponsorProfile only
- "Partner Sensitive Coordinator" — same for PartnerProfile only
- A user who administers all three holds all three roles. Someone who only oversees sponsor data holds only the sponsor coordinator role.

Option 2 — One unified "Sensitive Master Coordinator" role

- Single role granting Read=All, Edit=All, Create=Yes on all three sensitive entities
- Simpler administration, but no granularity — anyone with this role sees everything sensitive in the system

Why this matters

If responsibilities for client, sponsor, and partner sensitive data fall to different people at CBM (e.g., a Sponsorship Director should see sponsor data but not client data), only Option 1 supports that. Option 2 treats all sensitive data as a single trust tier — which works only if administration is genuinely centralized in one small group.

Cost of the recommended option

Option 1 is more roles to maintain (three instead of one). But since they're parallel and structurally identical, the maintenance burden is mostly conceptual, not practical.

Recommendation

Option 1. It mirrors the structural split you've already accepted (three teams, three entities) and follows least-privilege. A "Master Coordinator" is just a user who happens to hold all three of the granular roles — same effect, more flexibility.

Decision request

Approve **Option 1 (three coordinator roles)**, or argue for **Option 2 (one unified)**?

Implementation notes

- Three workflow rules — one per sensitive type. Each triggered on Company.assignedUser change, each updating the relevant SensitiveEntity.assignedUsers field. Independent of each other.
- The Data Integrity role and team stay as-is. They affect Company-level data only and don't intersect with sensitive entities.

After this decision, the model is complete

Once you settle the coordinator question, you have a fully-specified security architecture covering:

- One Company entity, viewable by all
- Three sensitive sub-entities, each hidden from non-authorized users via field-level link hiding
- Three "Authorized" roles (team-attached) for granular per-team ownership-derived access
- Three (or one) "Coordinator" roles for administration

- One Data Integrity role for company-data quality
- Three workflow rules for ownership-derived auto-sync
- One Standard User role as the universal baseline

That's the entire security model for this scope. Tell me Option 1 or 2 and I'll do a final consolidated spec.

Option 1. We already have a single role for users who need access to everything - Admin :)

12:26 AM

Good point — Admin is exactly the "see everything sensitive plus everything else" role, so a Master Coordinator would just be a less-privileged duplicate of Admin. Per-type Coordinators are right, and Admin covers the rest.

Here's the final consolidated spec for the entire security model.

CBM EspoCRM Security Architecture — Consolidated Spec

Summary

One Company entity is universally visible. Three sensitive sub-entities (ClientProfile, SponsorProfile, PartnerProfile) hold restricted data, hidden by default and only visible to users in the corresponding access team. Per-team users get automatic ownership-derived access to records for Companies they own, and can transfer access among their peers via a Team-scoped Assignment Permission. Per-type Coordinators manage their respective sensitive entities. A Data Integrity team handles Company-level data quality. Admin retains universal access.

Entity Setup

Existing entity

- **Company** — base entity, universally visible.

New sensitive sub-entities (each created the same way)

Entity	Relationship to Company	Link Field on Company	Link Field Back	Multiple Assigned Users
ClientProfile	one-to-one	clientProfile	company	enabled
SponsorProfile	one-to-one	sponsorProfile	company	enabled
PartnerProfile	one-to-one	partnerProfile	company	enabled

Teams

Team	Roles attached to team
Mentors	Client Sensitive Authorized

Team	Roles attached to team
Sponsor Coordinators	Sponsor Sensitive Authorized
Partner Managers	Partner Sensitive Authorized
Data Integrity Team	Data Integrity

Roles

1. Standard User (universal baseline — every user has this)

Scope Level Permissions:

Entity	Access	Create	Read	Edit	Delete	Stream
Company	enabled	yes	All	Own	No	All
ClientProfile	disabled	—	—	—	—	—
SponsorProfile	disabled	—	—	—	—	—
PartnerProfile	disabled	—	—	—	—	—

Field Level Permissions on Company:

Field	Read	Edit
clientProfile	No	No
sponsorProfile	No	No
partnerProfile	No	No

Permissions: Assignment Permission = No

2. Client Sensitive Authorized (attached to Mentors team)

Scope Level Permissions:

Entity	Access	Create	Read	Edit	Delete	Stream
ClientProfile	enabled	no	Own	Own	no	Own

Field Level Permissions on Company:

Field	Read	Edit
clientProfile	Yes	—

Permissions: Assignment Permission = Team

3. Sponsor Sensitive Authorized (attached to Sponsor Coordinators team)

Scope Level Permissions:

Entity	Access	Create	Read	Edit	Delete	Stream
SponsorProfile	enabled	no	Own	Own	no	Own

Field Level Permissions on Company:

Field	Read	Edit
-------	------	------

sponsorProfile	Yes	—
----------------	-----	---

Permissions: Assignment Permission = Team

4. Partner Sensitive Authorized (attached to Partner Managers team)

Scope Level Permissions:

Entity	Access	Create	Read	Edit	Delete	Stream
--------	--------	--------	------	------	--------	--------

PartnerProfile	enabled	no	Own	Own	no	Own
----------------	---------	----	-----	-----	----	-----

Field Level Permissions on Company:

Field	Read	Edit
-------	------	------

partnerProfile	Yes	—
----------------	-----	---

Permissions: Assignment Permission = Team

5. Client Sensitive Coordinator (assigned directly to user record)

Scope Level Permissions:

Entity	Access	Create	Read	Edit	Delete	Stream
--------	--------	--------	------	------	--------	--------

ClientProfile	enabled	yes	All	All	All	All
---------------	---------	-----	-----	-----	-----	-----

Field Level Permissions on Company:

Field	Read	Edit
-------	------	------

clientProfile	Yes	Yes
---------------	-----	-----

Permissions: Assignment Permission = All

6. Sponsor Sensitive Coordinator (assigned directly to user record)

Same structure as Client Sensitive Coordinator, scoped to **SponsorProfile** and the sponsorProfile link field.

7. Partner Sensitive Coordinator (assigned directly to user record)

Same structure as Client Sensitive Coordinator, scoped to **PartnerProfile** and the partnerProfile link field.

8. Data Integrity (attached to Data Integrity Team)

Scope Level Permissions:

Entity Access Create Read Edit Delete Stream

Company enabled not-set not-set All not-set not-set

Field Level Permissions: none.

Permissions: Assignment Permission = No (job is data quality, not ownership management).

User Assignment Patterns

User type	Direct role assignments	Team memberships
Standard staff	Standard User	(none)
Mentor	Standard User	Mentors
Sponsor Coordinator	Standard User	Sponsor Coordinators
Partner Manager	Standard User	Partner Managers
Data Integrity volunteer	Standard User	Data Integrity Team
Per-type Coordinator	Standard User + the relevant Coordinator role	(the corresponding "Authorized" team optional, since Coordinator already grants superset access)
Multi-role user (e.g. Mentor + Partner Manager)	Standard User	Both relevant teams
Admin	Admin	(n/a — bypasses ACL)

Workflow Rules

Three independent workflow rules, one per sensitive type. Each rule:

- **Triggered on:** Company.assignedUser change (and on creation of a new SensitiveEntity-to-Company link)
- **Action:** Look up the related SensitiveEntity (if exists). Add the current Company.assignedUser to the SensitiveEntity.assignedUsers field.
- **Behavior:** Additive only — never removes a user. This is Option A: previous owners stay until a Coordinator (or the user themselves, within their team-scoped Assignment Permission) removes them manually.

Workflow rule	Triggered by	Updates
Sync Mentor to ClientProfile	Company.assignedUser change	ClientProfile.assignedUsers (add)
Sync Sponsor Coordinator to SponsorProfile	Company.assignedUser change	SponsorProfile.assignedUsers (add)
Sync Partner Manager to PartnerProfile	Company.assignedUser change	PartnerProfile.assignedUsers (add)

Coordinator Stale-Access Dashlets

Each Coordinator role has a corresponding list view / dashlet showing SensitiveEntity records where assignedUsers contains a user who is no longer the related Company.assignedUser. Used for periodic manual cleanup of access lists.

Test Plan

Test users (none with admin):

- `test_standard` — Standard User only.
- `test_mentor` — Standard User + Mentors team. Set as Company.assignedUser on at least one Company that has a ClientProfile, and not on a different one that also has a ClientProfile.
- `test_dual_mentor_partner` — Standard User + Mentors team + Partner Managers team.
- `test_client_coord` — Standard User + Client Sensitive Coordinator role.
- `test_data_integrity` — Standard User + Data Integrity Team.

For each, log in via incognito window and verify:

- The correct sensitive panels appear (or don't) on Company detail views.
- The correct records appear in those panels.
- Editing of data fields is permitted where expected.
- `assignedUsers` editability matches the role's Assignment Permission.
- Stale-access dashlets show appropriate records for Coordinator users.

Status

The security model is fully specified. Nothing is left ambiguous.

Possible next steps, in priority order:

1. **Capture in the CBM How-To Guide** as a complete "Security Architecture" section with three sub-sections (Standard Roles, Sensitive Sub-Entity Pattern, Workflow Rules) and the test plan as an appendix.
2. **Draft the YAML representation** for CRM Builder — this entire spec is concrete enough to encode declaratively.
3. **Build the test environment** in EspoCRM and run the test plan to verify behavior matches the spec.

You pulled the brakes on #1 earlier, so I'm holding. Tell me which (if any) you want to proceed with.

I would like to capture the detailed explanation of espo security in the ESPO How To Guide. But then we need a section on the CBM specific implementation of security. That way a new user can learn what we learned together here today. Does the YAML representation support security setup?

12:32 AM

Architected two-section How-To structure and identified repository verification needs
Architected two-section How-To structure and identified repository verification needs

Good plan — two sections with two distinct audiences and purposes. Section A teaches the conceptual model so any reader can understand EspoCRM security; Section B shows how CBM specifically applies it. A new admin landing on the How-To learns the framework first, then sees the implementation.

Proposed structure

Section A — EspoCRM Security: How It Works

Generic content, transferable to any EspoCRM deployment. Reads as a primer for someone new to the system.

1. The access control question and the Admin bypass
2. The three building blocks: Users, Teams, Roles
3. The action × level matrix (Create / Read / Edit / Delete / Stream × All / Team / Own / No)
4. Field Level Security — what it can and can't do (role-based, never per-record)
5. Assignment Permission — the often-overlooked third axis
6. Multiple Assigned Users vs Collaborators — when each fits
7. Role merging — the additive, most-permissive-wins rule
8. Roles via team attachment vs direct user assignment
9. Hiding relationship panels via field-level access on link fields
10. The Sensitive Sub-Entity Pattern — pulling it all together as a reusable design pattern

Section B — CBM Security Implementation

CBM-specific. Reads as the applied form of Section A's concepts. References back to Section A throughout.

1. Architecture summary
2. Entities and relationships (Company, ClientProfile, SponsorProfile, PartnerProfile)
3. Teams (Mentors, Sponsor Coordinators, Partner Managers, Data Integrity Team)
4. Roles — all eight, with full configuration tables
5. User assignment patterns
6. Workflow rules
7. Coordinator stale-access dashlets
8. Test plan

Please upload `CBM-EspoCRM-HowTo.docx` and I'll do a surgical insert: add both sections in the right place in the existing structure, add a change-log entry, and update the "Last Updated" timestamp. I'll show you the proposed text before I regenerate the .docx so you can review before commit.

On the YAML question

Honestly: I don't know off the top of my head whether the CRM Builder YAML schema currently supports security configuration (roles, teams, field-level permissions, Assignment Permission,

workflow rules). It's plausible, but I'd need to inspect the repository to give you an accurate answer — specifically the YAML schema definition, any security-related PRDs, and any sample YAML files that include security blocks.

Per your standing instructions, before I do any repo work I should confirm: **which CLAUDE.md should I read for the YAML investigation — the one in crmbuilder, or do you want to defer the YAML question until after the How-To work is done?**

Yes. Defer the YAML Question until next action. It deserves a focused discussion.

12:35 AM